# Highlights

## Chance Constraints as Effective Deadlines for Fast Replanning of UAV Fleets Under Uncertainty

Cao-Phuc Ha, Vu Minh Anh Nguyen, Duong Thanh Tran, Phu-Nguyen Le

- Chance constraints on task finish times reduce to closed-form effective deadlines
- A cost-aware fallback attempts a given-up task only if its expected value pays off
- Seven studies on held-out missions, each with a protocol frozen before its runs
- LTCC served 5.07 points more reward than the nominal model and 10.0% more than CBBA
- In PX4 software-in-the-loop flights, the fallback raised the utility by 56%

# Chance Constraints as Effective Deadlines for Fast Replanning of UAV Fleets Under Uncertainty

Cao-Phuc Ha[a], Vu Minh Anh Nguyen[a], Duong Thanh Tran[b] and Phu-Nguyen Le[c,*]

[a]Department of Artificial Intelligence, FPT University, Ho Chi Minh City, 700000, Vietnam
[b]Department of Integrative Biotechnology, College of Biotechnology and Bioengineering, Sungkyunkwan University, Suwon, 16419, Republic of Korea
[c]Faculty of Electrical and Electronics Engineering, Vietnam Aerospace University, Ho Chi Minh City, 700000, Vietnam

ARTICLE INFO

*Keywords*:
Multi-UAV systems
Task allocation
Dynamic replanning
Chance constraints
Execution uncertainty
Software-in-the-loop simulation

ABSTRACT

Multi-UAV replanners usually plan for nominal execution, so wind, speed and service-time errors make the tasks near the end of a route late. This paper proposes LTCC, a replanner built on the observation that, in the tested missions, the spread of the finish-time error was well approximated by an affine function of the lead time. With this error model, a chance constraint on every task reduces exactly to an effective deadline, so fast routing heuristics can be reused unchanged. A cost-aware fallback attempts a task that the constraint gives up only when its probability of meeting the deadline outweighs the cost of a late arrival, in closed form and without cost-specific tuning. Seven studies on held-out missions, each with a protocol fixed in advance, compared LTCC with time buffers, moment propagation and the consensus-based bundle algorithm (CBBA). LTCC served 5.07 percentage points more reward than the nominal model and 10.0% more than CBBA. With a late cost of one task reward, the cost-aware fallback scored 2.42 points above the nominal fallback. In PX4 software-in-the-loop flights, the fallback raised the utility by 56%.

## 1. Introduction

Teams of unmanned aerial vehicles (UAVs) are used for inspection, delivery and monitoring. During a mission, new tasks appear, vehicles fail and deadlines are moved forward, and a central planner has to repair the routes of the whole fleet within a fraction of a second. Multi-robot task allocation and dynamic vehicle routing offer many methods for this repair [1, 2, 3, 4]. Auction methods such as the consensus-based bundle algorithm (CBBA) [5] and metaheuristics such as adaptive large neighbourhood search (ALNS) [6] find good plans within such short budgets.

Most of these replanners plan for nominal execution. When a UAV meets wind, flies slower than commanded or stays longer at a task, the slack of its plan is used up by errors that the planner does not see, and the tasks near the end of a route are served late. Stochastic and robust routing account for such errors [7, 8, 9, 10], and chance-constrained allocation bounds the probability of failure [11, 12]. These models are usually solved by sampling or exact optimisation, which takes longer than the time between two events. A time buffer in the schedule [13] is fast, but a constant buffer treats every task alike, although the error grows with the time until a task is reached. Work on anytime planning treats the computation time itself as the quantity to control [14, 15, 16], and it is not clear whether the planning model or the control of computation matters more when execution is uncertain. A further question concerns the tasks that a chance constraint gives up. Whether such a task is still worth attempting depends on what a late arrival costs. Soft time windows price lateness inside the search [17], and a buffer cannot answer the question at all, because it carries no probability of meeting the deadline.

This paper proposes LTCC (lead-time-scaled chance constraints), a replanner that keeps the fast heuristics of the nominal model and changes only the deadlines that they check. On development flights, the spread of the finish-time error was well approximated by an affine function of the lead time, consistent with wind and airspeed errors that act over a whole leg. With this error model, a chance constraint on the finish time of every task becomes an effective deadline in closed form. A cost-aware fallback uses the probability of meeting the deadline to decide which of the given-up tasks are still worth attempting, and it also takes the form of one deadline per task.

The contribution is threefold. The first is the planning model. A chance constraint on every task reduces exactly to an effective deadline (Proposition 1), and the cost-aware fallback attempts a given-up task only when its expected contribution is not negative (Proposition 2). It reuses the safety factor and the error model of the planning model and adds no parameter that has to be tuned for the cost of a late arrival, whereas a time buffer has to be tuned again for every such cost. Gaussian chance constraints, feasibility checks under uncertainty inside routing heuristics [8, 18], ALNS and CBBA are not new, and the contribution lies in the interface between them and in the decision rule built on it. The second is an evaluation in seven studies on held-out missions, each with a protocol fixed before its runs. It compares LTCC with re-implemented baselines from the literature, namely time buffers, moment propagation [8], stopping rules from metareasoning [19, 14, 15] and CBBA [5], and it separates the effects of the planning model, the fallback, the

*Corresponding author.
caophucai@gmail.com (C. Ha); anhnvmse203412@fpt.edu.vn (V.M.A. Nguyen); duongtt@skku.edu (D.T. Tran); nguyenlp@vaa.edu.vn (P. Le)
ORCID(s): 0009-0004-7024-5166 (C. Ha); 0009-0007-8206-3329 (V.M.A. Nguyen); 0009-0003-5096-8167 (D.T. Tran); 0000-0001-6773-6336 (P. Le)

stopping rule and the trigger. The third is a software-in-the-loop evaluation with PX4 autopilots [20] in Gazebo [21] and ROS 2 [22], on missions rescaled so that the decision problem stays the same.

In the kinematic tier, LTCC served more reward than the nominal model, a constant buffer and a conventional planner on three independent sets of held-out seeds, and 10.0% more than CBBA. A tuned proportional buffer and moment propagation, whose margins also grow with the lead time, reached a similar utility, so the shape of the margin carries most of the gain. When a late arrival cost one task reward, the cost-aware fallback scored above the nominal fallback and above competitors tuned for that cost. In software-in-the-loop flights, the fallback raised the utility by 56%, and the kinematic tier predicted the outcome of each flight comparison.

Section 2 reviews related work and Section 3 states the problem. Section 4 presents LTCC, Section 5 the evaluation and Section 6 the results, which Section 7 discusses.

## 2. Related work

### 2.1. Dynamic task allocation and routing

Multi-robot task allocation [1, 23] and dynamic vehicle routing [2, 3] reassign tasks when information changes, and recent reviews cover their use for UAVs [4, 24]. The planning problem considered here is a prize-collecting routing problem with time windows [25, 26], and large neighbourhood search has been adapted to dynamic routing with time windows [27]. Auction methods such as CBBA reach conflict-free assignments quickly [5] and have been extended to dynamic task arrivals with time windows [28], and learning-based schedulers combine reinforcement learning with stable matching for UAV-assisted mobile computing [29]. Recent multi-UAV work handles stochastic task completion with hierarchical policies [30], uncertain execution times with dynamic time slices [31], intermittent communication with uncertainty-gated allocation [32], and events with assignment-preserving or local repair [33, 34]. Event-triggered and periodic allocation have been compared for multi-UAV systems [35], and online allocation architectures have been demonstrated in PX4 and ArduPilot software-in-the-loop simulation [36, 37]. Table 1 places these approaches next to LTCC. Most of the multi-UAV approaches replan online, but few combine deadlines, execution uncertainty and a cost of lateness, and only the metareasoning work treats the computation time as a decision.

### 2.2. Uncertainty in routing and allocation

Stochastic vehicle routing treats travel and service times as random variables, including correlated and time-dependent travel times [9, 18], and multi-robot planning under uncertainty treats the durations of actions as random [41]. A priori routes with recourse protect time windows against stochastic service times [42], robust formulations protect them against uncertain travel times [7], and self-imposed time windows add buffer time to the schedule [13]. Li et al. [43] modelled routing with time windows and stochastic travel and service times both as a chance-constrained program and as a stochastic program with recourse. Ehmke et al. [8] propagate the mean and variance of the arrival time along a route, with Clark's approximation of the maximum of two normal variables [44], and use the result as a feasibility check inside routing heuristics. Soft time windows instead charge a penalty for late service in the objective [17]. For UAVs, Gao et al. [12] solve a distributionally robust chance-constrained task assignment as a mixed-integer program, Wang et al. [40] estimate the effect of uncertain flight times on completion times, and Zhou and Tokekar [38] select risk-averse multi-robot assignments with the conditional value at risk. Drone delivery has been planned with speed-dependent energy [45], robust flight times [39] and ambiguity sets built from wind observations [10]. Most of these formulations are solved with mixed-integer, sampling-based or greedy methods over a planning horizon rather than within the few milliseconds available between two events.

LTCC differs in four respects. Its error model is fitted to the finish times observed in closed loop. It runs between events within milliseconds. It adds a fallback for the tasks that a hard chance constraint gives up. And it prices lateness only in this fallback, through a break-even probability that again yields a deadline, so the search keeps its objective and its feasibility checks. The closest use of a probability as an acceptance test is the uncertainty-gated allocation of Qian and Fu [32], which acts on single agent-task pairs of a one-task-per-UAV matching, whereas LTCC acts on the deadline of every task along a route.

### 2.3. Metareasoning and anytime algorithms

Anytime algorithms and metareasoning decide how long a planner should compute [14, 16], from performance profiles [19], online performance prediction [15] or learned stopping policies [46]. Situated temporal planning and deadline-aware metareasoning account for the time that passes during planning [47, 48]. Two of these stopping rules serve as baselines, so that the value of controlling computation can be compared with the value of the planning model.

## 3. Problem formulation

A central planner is called whenever a replanning trigger fires. Each UAV keeps the task that it is currently flying to or serving, which fixes when and where it becomes available again and how much energy it has left. Each open task has a location, a release time, a deadline by which its service must end, a service duration and a reward. A plan assigns an ordered sequence of open tasks to each UAV, and tasks may remain unassigned, so the planning problem is a prize-collecting routing problem with time windows [25, 26]. Arrival and finish times along a route follow the usual rules for time windows [49]. A route is feasible if every service ends before its deadline and the UAV returns to the depot with 10% of its energy. The objective of a plan is the reward

Table 1
Comparison with related approaches.

| Work | Problem | TW | Unc. | Late cost | Online | Trigger | Stop | SITL | Solver or architecture |
|---|---|---|---|---|---|---|---|---|---|
| Taş et al. [17] | VRP, soft time windows | ✓ | ✓ | ✓ | - | - | - | - | tabu search |
| Ehmke et al. [8] | VRPTW, service levels | ✓ | ✓ | - | - | - | - | - | heuristic, moments |
| Zhou and Tokekar [38] | multi-robot assignment | - | ✓ | - | ✓ | ○ | - | - | greedy, CVaR |
| Cheng et al. [10] | drone scheduling, wind | ✓ | ✓ | ✓ | ○ | - | - | - | DRO, branch and cut |
| Bruni et al. [39] | drone location routing | - | ✓ | - | - | - | - | - | robust optimisation |
| Gao et al. [12] | UAV assignment | ✓ | ✓ | - | - | - | - | - | MILP, DRCC |
| Wang et al. [40] | UAV task planning | ✓ | ✓ | - | - | - | - | - | completion-time estimate |
| Choudhury et al. [30] | MRTA, time windows | ✓ | ✓ | - | ✓ | - | - | - | hierarchical |
| Huang et al. [31] | multi-UAV allocation | - | ✓ | - | ✓ | ○ | - | - | - |
| Calvo and Capitán [36] | long-endurance UAV MRTA | ○ | - | - | ✓ | - | - | ✓ | centralised |
| Hwang et al. [33] | multi-UAV replanning | ✓ | - | - | ✓ | - | - | - | - |
| Shen et al. [34] | UAV swarm allocation | ○ | - | - | ✓ | - | - | - | edge |
| Qian and Fu [32] | multi-UAV allocation | ✓ | ✓ | - | ✓ | - | - | - | distributed |
| Cong et al. [35] | multi-UAV allocation | ✓ | - | - | ✓ | ✓ | - | ○ | - |
| Ali et al. [37] | multi-UAV allocation | - | - | - | ✓ | - | - | ✓ | decentralised |
| Svegliato et al. [15] | anytime metareasoning | - | - | - | - | - | ✓ | - | - |
| LTCC (proposed) | multi-UAV replanning | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | centralised |

TW: time windows or deadlines; Unc.: execution or duration uncertainty in the planning model; Late cost: a cost of late service in the planning decision; Online: replanning during the mission; Trigger: trigger policies compared; Stop: computation time treated as a decision; SITL: validation with an autopilot in software-in-the-loop or physics simulation. ✓: yes; ○: partly; -: no or not reported.

of the tasks served on time, minus 0.05 per kilometre flown and 0.3 per task that moves to another UAV than in the previous plan. The two small penalties only guide the search, against detours and against plans that change more than needed between calls. The evaluation uses the utility defined below, which counts served reward only.

Computation takes mission time. A plan computed for some milliseconds is released a multiple of this time later, and this latency factor emulates a planning computer that is slower than the evaluation computer. An idle UAV waits for the new plan, so its route starts later, and the plan reaches the vehicles only after a dissemination latency that the planner does not control.

The planner uses a nominal model with straight legs at the cruise speed of each UAV and nominal service durations. The executed finish time of a task differs from its planned finish time by an execution error caused by acceleration, wind, speed tracking and variable service times, and a task yields its reward only if its service ends before the deadline. As in dispatching practice, a UAV skips a task at departure if its nominal estimate already misses the deadline, and the task returns to the pool. The utility of a mission is the reward of the tasks served on time divided by the total reward. Three replanning triggers are considered. The event trigger replans on task arrivals, UAV failures and deadline changes, the periodic trigger also every 60 s, and the deviation trigger also when a finish time deviates from the plan by more than a threshold. During planning, every UAV finishes its committed task and then waits, and events are queued.

## 4. Proposed method

LTCC takes four decisions in every replanning call (Fig. 1). The planning model decides which plans are acceptable under the calibrated execution error. The cost-aware fallback decides which of the tasks that this model gives up are still worth attempting. The stopping rule decides when to publish, and the trigger decides when to replan. The first two decisions are the contribution of this paper, and both reduce to one deadline per task, so the repair and the search of the nominal model run unchanged.

Fig. 1 shows the architecture in three panels. Panel A is done once, before any mission: the error model, the safety factor and the gain predictor of the stopping rule are fitted on the finish-time errors of development missions, per disturbance level. Panel B shows one replanning call in five steps. An event, such as a new task or a failed UAV, or the periodic trigger starts the call on a snapshot of the open tasks, the UAV states and the previous plan (step 1). The calibrated error model gives a margin that grows with the lead time (step 2), and this margin turns the deadline of every task into an effective deadline (step 3). The cost-aware fallback compares the probability of meeting the deadline of a given-up task with its break-even probability and keeps the task only if the attempt pays off (step 4). Repair and ALNS then search on these deadlines until the stopping rule publishes the best plan (step 5). Panel C shows the execution in tier B or tier C, whose telemetry starts the next call. Steps 2 to 4 are the proposed components, and the other steps are standard. Two earlier variants of the same replanner, AnytimeSwarm and AnytimeSwarm-R, keep the nominal deadlines and differ only in their stopping rule, and they serve as baselines.

### 4.1. Repair and anytime search

The previous plan is restricted to the open tasks and kept as far as its routes remain feasible. Tasks of failed UAVs and new tasks are inserted by regret-2 insertion [6], which yields a publishable plan within milliseconds. ALNS then improves the plan with the standard destroy and repair operators, adaptive operator selection and simulated-annealing

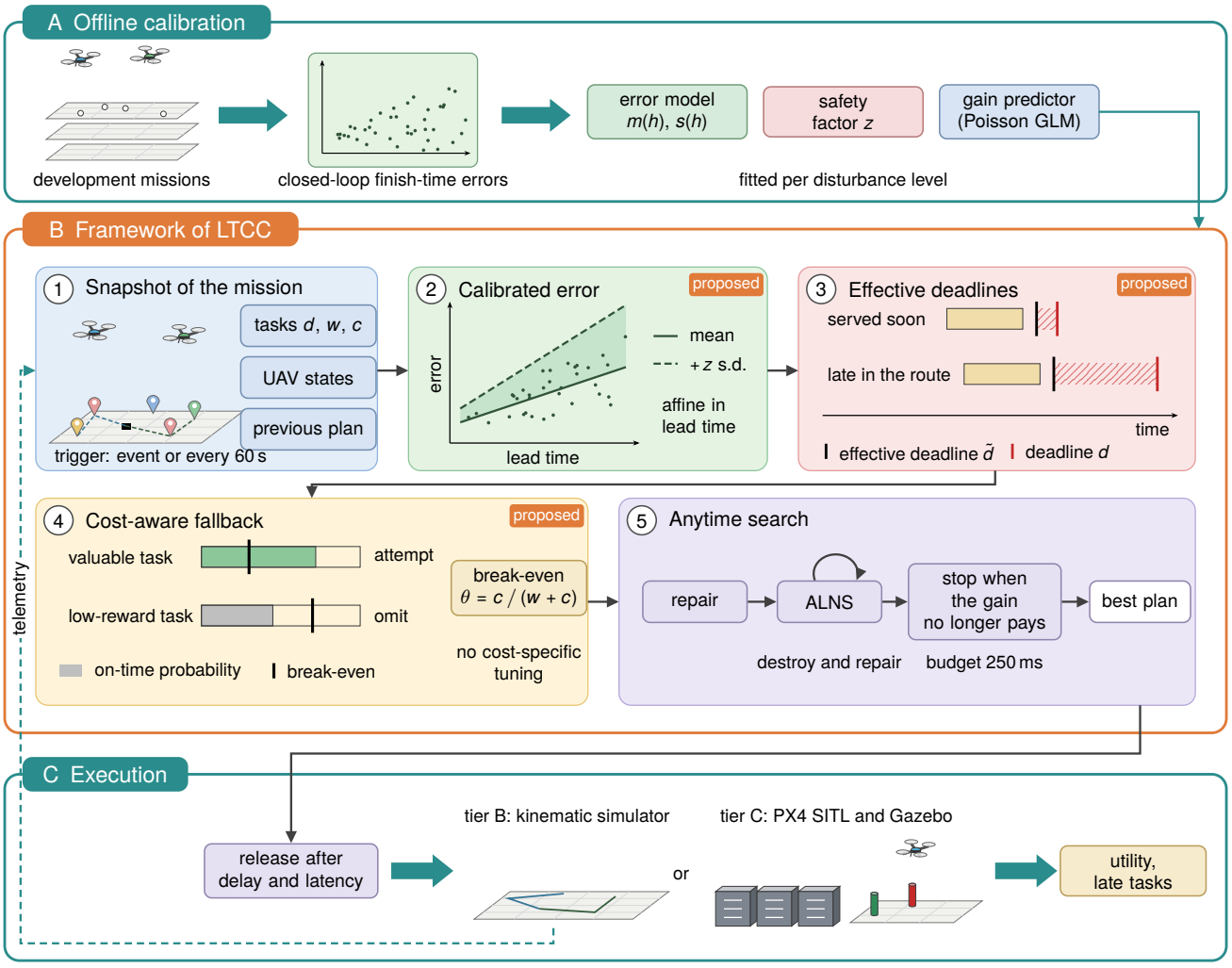



**Figure 1:** Architecture of LTCC. (A) Offline calibration on development missions. (B) The five steps of one replanning call, with the proposed components marked. (C) Execution of the plan, whose telemetry starts the next call.

acceptance [6, 50]. Every intermediate plan is feasible, so the best plan found so far can be published at any time. All variants share the random seed and therefore follow the same search trajectory, so that they differ only in their deadlines or in the moment at which they stop.

## 4.2. Calibrated finish-time error

The finish-time error of a task is modelled as normal, with a mean $m$ and a standard deviation $s$ that are affine in its lead time $h$, the time from the planning snapshot to the planned finish,

$$m(h) = b_0 + b_1 h, \qquad s(h) = s_0 + s_1 h. \tag{1}$$

The four coefficients are fitted per disturbance level on development missions (Supplementary Table S2). They are estimated from pairs of lead time and error observed in closed loop, where the error is the actual minus the planned finish of an executed task. These pairs therefore already contain the remaining time of the task that a UAV is serving at the snapshot, the delay until the plan is released and the variation of the start of each route. Both the mean and the spread grow with the lead time, so the nominal model is optimistic, and more so for the tasks late in a route.

An online variant removes the need to know the disturbance level. It starts from one error model fitted on the development missions of both levels, given the weight of 50 pairs, and refits Eq. (1) at every call with the pairs of the tasks completed so far in the mission. Its safety factor stays fixed at 0.5. Section 6.2 compares it with the model fitted for the known level.

Fig. 2 shows why an affine spread is a good approximation. Panel (a) plots the standard deviation of the error against the lead time. A model with independent increments per unit of time, the usual assumption when variances are added along a route, gives a square-root curve and underestimates the spread at long lead times. At about 514 s under high disturbances, it predicts 262 s, whereas 372 s are observed and the affine model of Eq. (1) predicts 380 s. The affine model also fits better by the Akaike information criterion, by 859 and 910 points under mid and high disturbances,

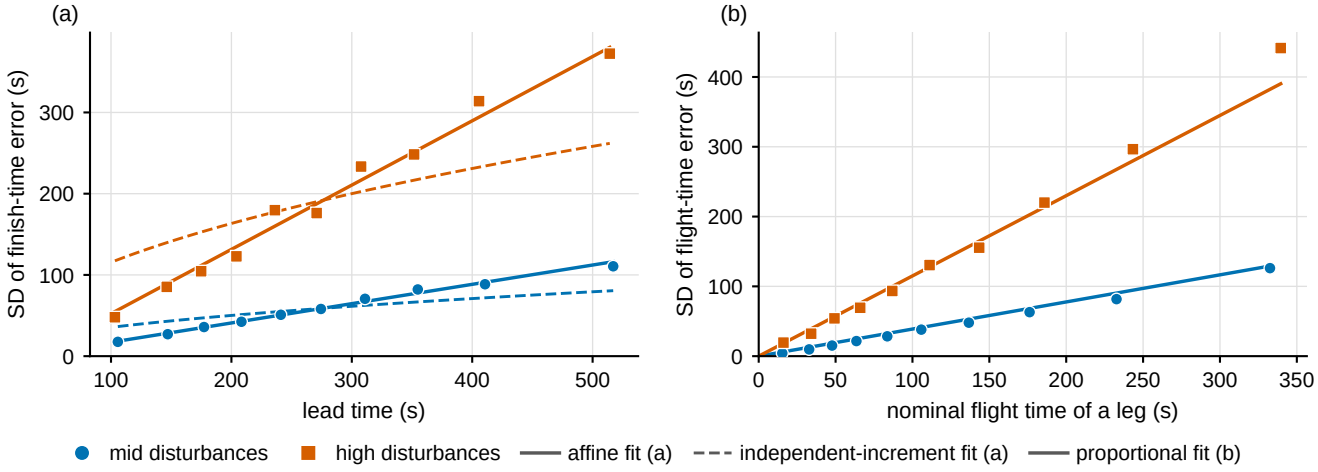



**Figure 2:** Finish-time error on development missions. (a) Standard deviation against lead time, with the affine and the independent-increment fit. (b) Standard deviation of single legs against their nominal flight time, with a proportional fit. Points are decile bins.

and in a five-fold cross-validation with missions as groups. Panel (b) suggests a reason. The error of a single leg grows in proportion to its duration, consistent with wind and an airspeed error that act over the whole leg, while the errors of successive legs are close to independent, with correlations of -0.20 and -0.05. With independent legs, the spread of a route grows with the root of the summed squared leg durations rather than with their sum, so the affine form does not follow from this structure in general. It is used here as an empirical approximation, which the data of panel (a) support over the observed range of lead times and mission configurations.

### 4.3. Effective deadline

LTCC accepts a planned finish time $f$ only if the deadline $d$ still holds after adding the mean error and $z$ standard deviations, where the safety factor $z$ sets the target probability of being on time. Since the margin is linear in the finish time itself, this chance constraint has a closed form,

$$\begin{aligned} & f + m(f - t_0) + z\, s(f - t_0) \le d \\ \iff\ & f \le \tilde{d} = t_0 + \frac{d - t_0 - b_0 - z s_0}{1 + b_1 + z s_1}, \end{aligned} \tag{2}$$

where $t_0$ is the time of the snapshot. The repair and the search check the effective deadline $\tilde{d}$ instead of $d$, while the executor and the reward keep the true deadline. The margin is small for tasks served soon and large for tasks late in a route, and it costs one division per task.

Proposition 1. Let $s_0, s_1 \ge 0$ and $1 + b_1 + z s_1 > 0$. Under the error model of Eq. (1), a task planned to finish by its effective deadline meets its true deadline with probability at least $\Phi(z)$, where $\Phi$ is the standard normal distribution function, provided that its route starts as planned. Without execution errors, the effective deadline equals the deadline.

The proof is given in Supplementary Section S1. The condition on the start of the route refers to the error model. Because the model is fitted on errors observed in closed loop, it already covers the variation of the time at which a UAV becomes free and the delay of the plan, as far as they occurred on the development missions. The guarantee is therefore only as good as the calibration of the error model, which Section 6.2 and Supplementary Section S11 check on held-out missions. The planner caps $\tilde{d}$ at $d$, so that no task is planned beyond its deadline. With $z = 0$, the model corrects the mean error only. Larger values protect more tasks against lateness but leave fewer tasks feasible, and $z$ is chosen per disturbance level on development missions.

### 4.4. Fallback for given-up tasks

A hard chance constraint has a side effect when the time windows are short compared with the spread of the error. A task that no UAV can finish by its effective deadline, even on a direct flight, is never attempted, although a UAV may still serve it in time with a fair probability and may have nothing better to do. The nominal fallback plans such a task against its true deadline, provided that some UAV can still reach it in time. All other tasks keep their effective deadlines and the guarantee of Proposition 1. This fallback was designed after the results of study 2 and tested on new seeds in studies 3 and 4.

### 4.5. Cost-aware fallback

The nominal fallback is right only if a late arrival costs nothing. In practice, a late arrival wastes flight time and energy, and a customer may have left or a commitment may be broken. Let a late arrival at task $j$ cost $c_j$, let its reward be $w_j$, and let $p$ be the probability that an attempt meets the deadline. The attempt is worth making if its expected value $p\, w_j - (1 - p)\, c_j$ is not negative, that is, if

$$p \ \ge\ \theta_j = c_j / (w_j + c_j). \tag{3}$$

The late cost is measured in task rewards by a factor $\lambda$. It can be proportional to the reward of the task or a fixed amount of $\lambda$ mean task rewards. With a fixed cost, valuable tasks have a lower break-even probability and are attempted more readily.

The cost-aware fallback turns this rule into a deadline. By Proposition 1, the break-even probability corresponds to the safety factor $z_j = \Phi^{-1}(\theta_j)$, and Eq. (2) evaluated at $z_j$ gives the latest finish at which an attempt still pays off. Each task is protected at the larger of $z$ and $z_j$. A task that no UAV can reach in time under this protection is planned against its effective deadline at $z_j$, capped at the true deadline, and it is given up only if it cannot meet this deadline either. Every task thus keeps a closed-form deadline, and the late cost changes only its safety factor.

Proposition 2. Under the conditions of Proposition 1, the cost-aware fallback (i) equals the nominal fallback when a late arrival costs nothing, (ii) plans only tasks whose expected contribution, the reward times the probability of being on time minus the late cost times the probability of being late, is not negative if their routes start as planned, and (iii) never loosens a planning deadline when the late cost grows.

The proof is given in Supplementary Section S1. The criterion in part (ii) is local. It prices each attempt on its own and ignores the delay that a fallback task causes to the later tasks on its route, a cost that grows with the number of these tasks and weighs more when lateness is expensive. The rule adds no parameter that depends on the late cost. It reuses the safety factor and the error model of the planning model, which are calibrated once on development missions, and the late cost itself is a property of the application. It needs only a probability of meeting the deadline, so it applies equally to moment propagation [8], which accepts a given-up task if its mean finish plus $z_j$ standard deviations meets the deadline. A time buffer has no such probability, so its fallback has to be tuned again for every late cost. The cost-aware fallback was designed after the results of study 4 and tested on new seeds in studies 5 to 7.

### 4.6. Stopping rule and trigger

The stopping rule follows the earlier variant AnytimeSwarm-R. Every 5 ms, it compares the gain of searching 10 ms longer, predicted from the search history by a Poisson generalised linear model [51], with the cost of doing so. This cost is the expected value of the tasks that a later plan would lose, computed from the error model and the expected latency [49], plus a small price of computation (Supplementary Section S2). A hard budget of 250 ms bounds every call. LTCC replans on every event and at least every 60 s of mission time, a rolling-horizon policy in the terminology of dynamic routing [3].

## 5. Evaluation

### 5.1. Scenarios and kinematic tier

Missions are created by a seeded generator (Supplementary Table S1). The map is a square with a side of 10 km and the depot at its centre. Half of the tasks are clustered around three hotspots, and half of them are known at the start while the others arrive in batches. Time windows have a slack of 180 to 600 s and services last 30 to 90 s. Task rewards are 1, 2, 3 or 5 with probabilities 0.4, 0.3, 0.2 and 0.1, so the mean reward is 2.1. Three fleet scales with 5, 10 and 20 UAVs and 25, 50 and 140 tasks are combined with three event profiles, namely six arrival batches, three batches with two UAV failures, and three batches with four tightened deadlines.

The kinematic tier, called tier B, executes each published plan in continuous mission time. Legs follow a trapezoidal speed profile with an acceleration limit, the airspeed of each leg deviates from the nominal speed by a multiplicative error, a constant wind of random direction changes the ground speed, and service durations are scaled by a random factor. Table 2 lists the three disturbance levels and the two latency levels. The planner never sees these disturbances and receives the state of each UAV as telemetry. All random draws use common random numbers [52] keyed to the mission seed, the UAV and the task, so two planners that send the same UAV to the same task meet the same disturbance.

### 5.2. Software-in-the-loop tier

The software-in-the-loop tier, called tier C, flies missions with three X500 quadrotors, each controlled by its own PX4 v1.17 autopilot [20], in Gazebo Harmonic [21] on ROS 2 Jazzy [22]. Fig. 3 shows how a mission is prepared and executed. A tier C mission is a tier B scenario with three UAVs and fifteen tasks, rescaled by 0.1 in length and 0.25 in time. Speeds, energy, the distance price and the latency factor are scaled accordingly, so every dimensionless ratio of the decision problem is preserved. The mission manager runs the scenario timeline, builds the planning snapshot, calls the planner and releases the plan. The executor streams offboard setpoints to every autopilot at 10 Hz through the uXRCE-DDS bridge, flies each leg towards a target point that moves along the straight leg, and holds the UAV at the task for the service duration. The commanded ground speed follows the speed error and the scaled wind of tier B with the same random numbers, while acceleration, attitude dynamics, position tracking and communication come from the simulated vehicles. Gazebo runs in lockstep with PX4 at twice real time, and mission time is simulation time.

### 5.3. Compared methods

Table 3 lists the compared methods. All of them share the repair, the ALNS operators and parameters, the initial plan and the random seed, and every call has a hard budget of 250 ms. They differ in the planning model, the fallback, the rule that ends the search and the trigger. Unless stated otherwise, every method searches for a fixed 40 ms, the mean computation of the stopping rule on development data, and replans periodically.

The nominal model plans against the true deadlines. Besides 40 ms, it is run with regret-2 repair only, with 250 ms, with 1 ms per open task and with a rule that stops after 20% of the budget without improvement. The two stopping rules from the metareasoning literature use the same delay cost as LTCC and differ only in how they predict the gain of further search. The constant buffer, the simplest form of a time buffer in the schedule [13], subtracts 60 s from

**Table 2**
Disturbance and latency levels of the kinematic tier.

| Execution | Acceleration (m/s$^2$) | Speed c.v. | Wind (m/s) | Service c.v. | Latency | Median (s) | Log-s.d. | Loss / timeout |
|---|---|---|---|---|---|---|---|---|
| Low | 3.0 | 0.03 | 2 | 0.05 | Low | 0.2 | 0.5 | - |
| Mid | 2.0 | 0.07 | 5 | 0.15 | High | 2.0 | 0.8 | 5% / 5 s |
| High | 1.5 | 0.12 | 8 | 0.25 | | | | |

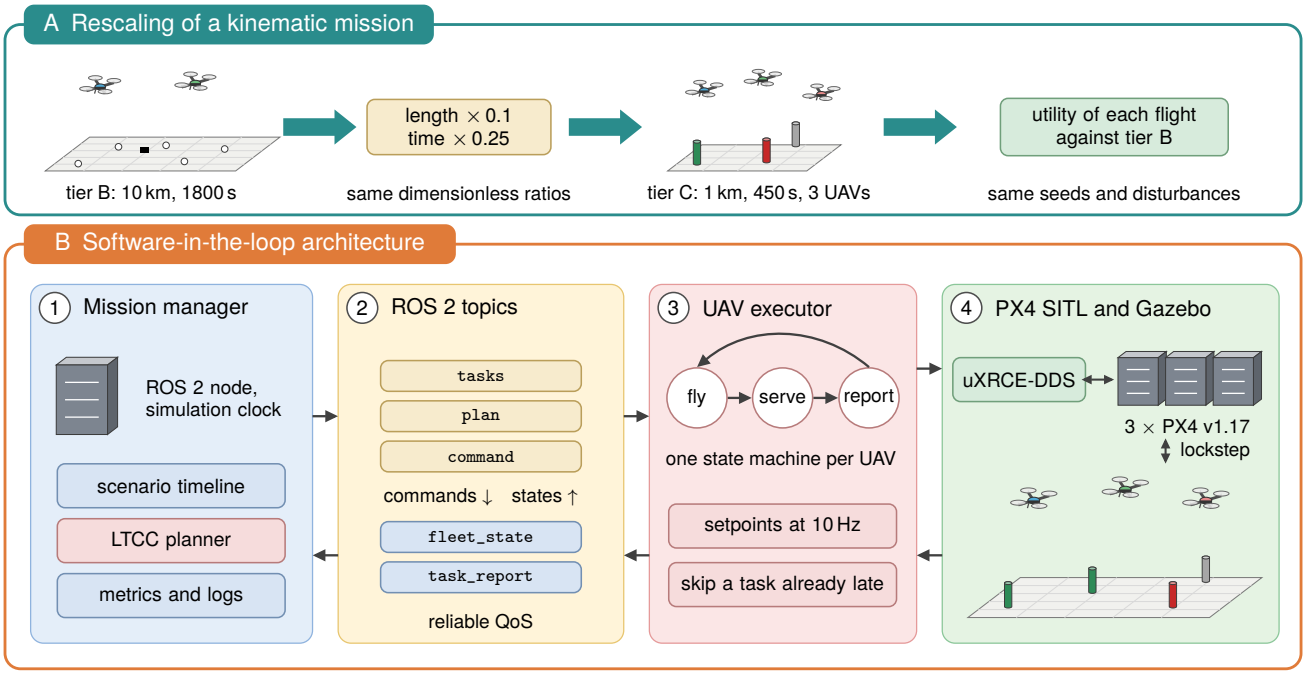



**Figure 3:** Software-in-the-loop tier. (A) Rescaling of a kinematic mission. (B) Software architecture.

**Table 3**
Compared methods.

| Method | Planning deadline | Fallback | Search | Studies |
|---|---|---|---|---|
| Nominal model [6] | deadline | none | repair only, or ALNS with a fixed, scaled or stagnation budget | 1-7 |
| Conventional planner | deadline, event trigger only | none | ALNS, 40 ms | 2-5 |
| AnytimeSwarm, AnytimeSwarm-R | deadline | none | stopping rule | 1, 2 |
| Performance profile [19, 14] | deadline | none | stopping rule | 2 |
| Online prediction [15] | deadline | none | stopping rule | 2 |
| Constant buffer [13] | deadline minus 60 s | none | ALNS, 40 ms | 2-5, 7 |
| Proportional buffer | deadline minus a tuned share of the lead time | none or nominal | ALNS, 40 ms | 4, 5, 7 |
| Moment propagation [8] | mean plus $z$ standard deviations along the route | none, nominal or cost-aware | ALNS, 40 ms | 4-7 |
| CBBA [5, 53] | deadline or LTCC deadlines | none or cost-aware | consensus | 6 |
| LTCC (proposed) | effective deadline, Eq. (2) | none, nominal or cost-aware | ALNS, 40 ms or stopping rule | 2-7 |

every deadline, the best of 30, 60 and 120 s on development missions. The proportional buffer subtracts a tuned share of the lead time, so it keeps the shape of the LTCC margin but replaces the calibration by one slope. Moment propagation follows Ehmke et al. [8] with leg parameters fitted on development flights. CBBA is re-implemented with the bid warping of Johnson et al. [53], which time windows require for convergence, on a fully connected network, and it pays for its planning time like every other method. Every model has one parameter per disturbance level, selected on development missions. For LTCC, the safety factor is 0.5 under mid and 0 under high disturbances without fallback, and 1 and 0.5 with fallback. Supplementary Section S4.2 gives the remaining parameters.

When a late arrival has a cost, a run is scored by

$$u_\lambda = U - \lambda\, n_{\text{late}}/n, \qquad u_\lambda^w = U - \lambda\, R_{\text{late}}/R, \qquad (4)$$

where $U$ is the utility, $n_{\text{late}}$ of the $n$ tasks are served late, and they carry the reward $R_{\text{late}}$ of the total reward $R$. The score $u_\lambda$ charges every late arrival $\lambda$ mean task rewards, and $u_\lambda^w$ charges $\lambda$ times the reward of the task. With $\lambda = 0$, both equal the utility. The competitors of the cost-aware fallback, namely the proportional buffer and LTCC without fallback, are tuned on development missions for every late cost and score, whereas the cost-aware fallback keeps the safety factor of the utility.

### 5.4. Studies and statistical protocol

The evaluation consists of seven studies. Each study has its own held-out seeds, a written protocol and frozen code, and each study after the first was designed after the held-out results of the earlier ones. Study 1 made only the stopping rule execution-aware. Study 2 compared LTCC without fallback with the baselines, study 3 added the nominal fallback, and study 4 compared planning models on the same search. Study 5 introduced the cost-aware fallback, study 6 compared it with CBBA and applied it to moment propagation, and study 7 tested the late-cost model of the rule (7a) and execution errors that the calibrated model does not describe (7b). Studies 5 and 6 ran the rule with a cost proportional to the reward but scored it with $u_\lambda$, which charges a fixed cost per late arrival. This mismatch between the cost model of the rule and the score was noticed after both studies had been frozen and run, so it could not be corrected without changing their protocols. Study 7a was designed to test it on new seeds. It runs both cost models, each scored with its own score. Study 7b draws speed and service errors from a Student-$t$ law, adds rare long delays, adds a persistent speed bias per UAV, or replaces the latency by a heavy-tailed Pareto delay. Tier C flights belong to studies 2 and 4, the latter called study 4C.

Development used seeds 0-9 in tier B and 0-2 in tier C. Before each study, the protocol with its hypotheses, the configurations, the models and the code were frozen with SHA-256 hashes and committed to the version history, and the held-out runs checked these hashes before starting. Studies 2 to 7 share one tier B matrix of two disturbance levels (mid and high), high latency, three latency factors (10, 50 and 200) and two time-window factors (original and halved). Each study uses 30 new held-out seeds. Every seed generates nine missions, one for each of the three fleet scales and three event profiles, so a study runs 270 missions in twelve configurations. Study 4C flies 36 missions under high disturbances with halved windows, where tier B predicted the largest effect, with a size computed beforehand and one interim analysis with O'Brien-Fleming bounds. A tier C run whose real-time factor leaves the range 1.6 to 2.2 is repeated once.

The difference between two methods is averaged per mission over the configurations and tested with a paired bootstrap [54] of 10 000 resamples over missions. The primary hypotheses of each study form one family, fixed in its protocol, and are corrected with the Holm procedure [55]. Non-inferiority uses a margin of 0.5 percentage points (pp) and is followed by a superiority test in a fixed sequence. All other comparisons are descriptive. Because the missions of one seed share their execution disturbances, every tier B primary test was repeated with a bootstrap over seeds (Supplementary Table S10), which changed only the outcome of H9.

## 6. Results

Table 4 lists the primary hypotheses of studies 2 to 7, as fixed in their protocols, with the observed differences and outcomes. Each row compares two methods on the score named in the fourth column, and a positive difference favours the first method. The subsections follow the order in which the studies were designed, so that each design decision can be traced to the result that motivated it.

### 6.1. Planning model and control of computation

Study 1 made only the stopping rule execution-aware and kept the nominal deadlines. In 34 560 tier B runs, AnytimeSwarm-R needed 15% less computation per call than AnytimeSwarm, but its advantage over ALNS with 40 ms, +0.32 [+0.01, +0.63] pp, was not significant after the Holm correction ($p = 0.128$). Periodic replanning raised the utility of every method by +1.29 to +1.56 pp (Supplementary Section S6). Study 2 therefore moved the error model from the stopping rule into the planning model.

Study 2 compared LTCC without fallback with the baselines in 48 600 tier B runs (Supplementary Table S6). LTCC reached the highest utility, 52.15%. Its advantage over the six baselines of H1 ranged from +1.35 to +2.87 pp, and over the conventional planner it was +4.44 [+3.82, +5.05] pp, or 9.3% (H2). It also served 49% fewer tasks late than ALNS with 40 ms. The stopping rules hardly mattered on the nominal model, with utilities between 49.28% and 49.66% although their computation ranged from 11.2 to 58.1 ms per call. With the effective deadlines, the stopping rule was equivalent to a fixed budget of 40 ms (H3). Its value lay in avoiding long budgets on slow planners: with a latency factor of 200, the full budget of 250 ms lost 23.42 tasks per mission to late plans. Without fallback, however, LTCC was

Table 4
Primary hypotheses and outcomes.

| Study | H | Comparison | Score | Difference (pp) | Outcome |
|---|---|---|---|---|---|
| 2 | H1 | LTCC, no fallback, vs. six baselines | $U$ | 1.35 to 2.87 | supported, 6 of 6 |
| | H2 | LTCC, no fallback, vs. conventional planner | $U$ | 4.44 | supported |
| | H3 | stopping rule vs. fixed 40 ms (equivalence) | $U$ | 0.22 | equivalent |
| | H4 | tier C: LTCC, no fallback, vs. nominal model | $U$ | −0.25 | not supported |
| 3 | H5 | LTCC, nominal fallback vs. no fallback | $U$ | 1.97 | supported |
| | H6 | LTCC, nominal fallback vs. constant buffer, high, windows halved | $U$ | 3.31 | supported |
| | H7 | LTCC, nominal fallback vs. nominal model | $U$ | 4.61 | supported |
| | H8 | LTCC, nominal fallback vs. constant buffer | $U$ | 3.59 | supported |
| 4 | H9 | LTCC vs. proportional buffer | $U$ | −0.38 | not supported |
| | H10 | LTCC vs. moment propagation (NI) | $U$ | −0.25 | NI not shown |
| | H11 | moment propagation: nominal fallback vs. no fallback | $U$ | 9.66 | supported |
| | H12 | LTCC vs. constant buffer | $U$ | 4.22 | supported |
| | H13 | LTCC vs. nominal model | $U$ | 5.07 | supported |
| 4C | H14 | tier C: LTCC vs. nominal model | $U$ | −0.57 | not supported |
| | H15 | tier C: LTCC, nominal fallback vs. no fallback | $U$ | 10.35 | supported |
| 5 | H16 | cost-aware vs. nominal fallback | $u_1$ | 2.42 | supported |
| | H17 | cost-aware fallback vs. tuned proportional buffer | $u_1$ | 0.90 | supported, sup. |
| | H18 | cost-aware fallback vs. LTCC, no fallback, tuned safety factor | $u_1$ | 1.78 | supported, sup. |
| | H19 | cost-aware fallback vs. nominal model | $u_1$ | 11.75 | supported |
| 6 | H20 | LTCC, cost-aware fallback vs. CBBA | $u_1$ | 12.77 | supported |
| | H21 | CBBA on LTCC deadlines vs. CBBA | $u_1$ | 12.12 | supported |
| | H22 | moment propagation: cost-aware vs. nominal fallback | $u_1$ | 2.71 | supported |
| | H23 | moment propagation: cost-aware vs. no fallback | $u_1$ | 3.30 | supported |
| 7a | H24 | fixed-cost vs. proportional rule | $u_1$ | 0.24 | not supported |
| | H25 | fixed-cost rule vs. tuned proportional buffer | $u_1$ | 1.34 | supported, sup. |
| | H26 | proportional rule vs. tuned proportional buffer (NI) | $u_1^w$ | 0.20 | NI shown |
| 7b | H27 | LTCC, nominal fallback vs. nominal model, four error departures | $U$ | 4.83 to 5.67 | supported, 4 of 4 |
| | H28 | cost-aware vs. nominal fallback, four error departures | $u_1$ | 3.60 to 3.87 | supported, 4 of 4 |

Differences are point estimates of the first method minus the second (ranges over the six baselines for H1 and over the four conditions for H27 and H28); the 95% paired bootstrap intervals of the tier B tests are given in Supplementary Table S10 and those of the tier C tests in Section 6.3. $U$: utility; $u_1$, $u_1^w$: scores of Eq. (4) with $\lambda = 1$. NI: non-inferiority with a margin of 0.5 pp; sup.: superiority tested after non-inferiority. Holm correction within each study. With a bootstrap over seeds, the proportional buffer became significantly better in H9; no other outcome changed. Study 1 is reported in Supplementary Section S6.

too cautious under high disturbances with halved windows, where it served −2.01 [−2.80, −1.21] pp relative to the constant buffer. Many effective deadlines then fell before the earliest possible arrival, and this result led to the fallback.

## 6.2. Margin shape and fallback

Study 3 tested the nominal fallback on new seeds. Its advantage over LTCC without fallback was +1.97 [+1.61, +2.33] pp (H5), and over the constant buffer in the regime where study 2 had lost it was +3.31 [+2.58, +4.03] pp (H6). Over all configurations, its advantages over ALNS with 40 ms and the constant buffer were +4.61 [+4.15, +5.06] and +3.59 [+3.17, +4.02] pp (H7, H8). The fallback has a price in punctuality, since a fallback task is planned against its true deadline and is often served late. It served 8.33 tasks late per mission, 46% more than LTCC without fallback.

Table 5 compares the planning models of study 4 on a third set of seeds. All models use the same search for 40 ms, so they differ only in the margin and the fallback. The columns give the utility over all configurations and per regime, the tasks served late per mission and the search iterations per millisecond. Three patterns stand out.

First, a margin that grows with the lead time is better than none or a constant one. LTCC served 4.22 pp more reward than the constant buffer (95% CI 3.74 to 4.68, H12) and 5.07 pp more than the nominal model (4.63 to 5.52, H13), that is, 8.5% and 10.4% more. The advantage held in each of the four regimes, and with repair only and no search it was still 3.33 pp. Over the three independent sets of seeds of studies 2 to 4, every advantage over the constant buffer, the nominal model and the conventional planner was significant (Supplementary Fig. S8).

Second, the three margins that grow with the lead time reached utilities within 0.38 pp of each other. The proportional buffer served 0.38 pp more than LTCC (H9) and moment propagation 0.25 pp more (H10), so the calibration adds little utility beyond a well-tuned slope. The closed form was the cheapest of the three, with 1.13 more search iterations per millisecond than moment propagation.

Third, every chance-constrained model needs the fallback. Without it, moment propagation reached the lowest utility of all models, 44.44%, 9.66 pp below its version with fallback (H11), while it served the fewest tasks late. A hard chance constraint on every task, however it is evaluated, thus gives up tasks that are still worth attempting. The error

Table 5
Study 4: utility (%) per planning model and regime.

| | | Mid | | High | | | |
|---|---|---|---|---|---|---|---|
| Planning model | All | ×1 | ×0.5 | ×1 | ×0.5 | Late | Iter./ms |
| Moment propagation | 54.10 | 73.06 | 48.64 | 57.24 | 37.48 | 7.36 | 12.22 |
| Moment propagation, no fallback | 44.44 | 70.89 | 38.98 | 48.71 | 19.17 | 1.62 | 15.97 |
| Proportional buffer | 54.23 | 73.60 | 48.25 | 57.17 | 37.91 | 7.13 | 13.72 |
| Constant buffer (60 s) | 49.63 | 69.46 | 43.72 | 51.68 | 33.66 | 6.52 | 14.75 |
| Nominal model | 48.78 | 66.73 | 44.89 | 49.79 | 33.71 | 11.22 | 13.00 |
| Conventional planner | 47.40 | 65.13 | 43.54 | 48.48 | 32.46 | 10.28 | 9.04 |
| LTCC, online calibration | 53.55 | 73.22 | 48.25 | 55.26 | 37.48 | 7.63 | 13.59 |
| LTCC | 53.85 | 73.07 | 48.05 | 57.01 | 37.27 | 7.54 | 13.35 |

Regimes: disturbance level and time-window factor, averaged over the latency factors. Models with a margin use the nominal fallback unless stated. Late: tasks served late per mission; Iter./ms: search iterations per planning millisecond. 270 missions, search budget 40 ms.

model also ranked the served tasks by risk. Under high disturbances, 76.9% of them were served on time against a predicted 71.3% (Supplementary Fig. S10).

The online variant, which does not know the disturbance level, served 0.30 pp less than the model fitted for the known level and still well above the nominal model (Table 5). This comparison was secondary, so the protocol did not test non-inferiority, but it indicates that the model can be calibrated during the mission.

### 6.3. Software-in-the-loop flights

In study 2, 120 held-out flights with mid disturbances and the original windows did not confirm the advantage of LTCC without fallback (H4, −0.25 [−4.02, +3.57] pp). Tier B predicted the utility of single flights closely ($r$ = 0.86) but had expected only +2.82 [−1.20, +6.62] pp on these missions (Supplementary Section S7.2). Study 4C was therefore placed in the regime with the largest predicted effect and flew 108 flights on 36 new missions. All flights ran at a valid real-time factor, with a mean of 1.96.

Fig. 4 shows one development mission flown with the graphical interface. Panels (a) and (b) show the Gazebo scene, in which every task is a pillar coloured by its outcome, and panels (c) and (d) the trajectories recorded by PX4. The nominal model sent a UAV to a task that it reached after the deadline, whereas LTCC served the same task on time with another UAV and then one more task. It served 7 of the 15 tasks on time against 6, with 0 instead of 1 late arrival. This pair of flights illustrates the mechanism and is not part of any test.

Fig. 5 summarises the held-out flights of study 4C. For each primary comparison, it shows the tier C result next to the tier B result on the same missions and the tier B estimate on development seeds that sized the study. The fallback carried over to the flights. With it, LTCC served 10.35 pp more reward than without it (95% CI 6.41 to 14.39, H15), 28.99% against 18.64%, or 56% more, and it was better in 28 of the 36 missions. Against the nominal model, it served 0.57 pp less (−3.49 to 2.29), so H14 was not supported. Tier B gave the same picture on the same missions, 0.22 pp less than the nominal model and 8.89 pp more than without fallback, and it predicted single flights with a correlation of 0.83. The gap to the development estimate therefore came from the sampled missions rather than from the transfer to the autopilot.

### 6.4. Cost of a late arrival

The utility counts only the tasks served on time, so a late arrival earns nothing and costs nothing. Re-scored with a late cost, the earlier runs showed that the nominal fallback pays off only when a late arrival costs less than about one task reward. Its advantage vanished at a late cost of 0.5 task rewards in study 3, 1.1 for moment propagation in study 4 and 1.5 in tier C (Supplementary Fig. S11). Study 5 tested the cost-aware fallback on new seeds.

Table 6 gives the mean score of each method for four late costs and, in the last column, the tasks served late at one task reward. The cost-aware fallback and the tuned competitors are run for the late cost of each column, so each column is a separate comparison. At one task reward, the cost-aware fallback scored 45.81% against 43.40% for the nominal fallback, 2.42 pp more (95% CI 1.81 to 3.02, H16), and it served 33% fewer tasks late. It was also 0.90 pp above the proportional buffer tuned for this cost (H17) and 1.78 pp above LTCC without fallback with a tuned safety factor (H18). Both competitors had been selected from sixteen and five candidates for this cost, whereas the cost-aware fallback kept the safety factor of the utility and had no cost-specific tuning.

Fig. 6 shows how these differences change with the late cost. The advantage over the nominal fallback grows with the cost, because the nominal fallback attempts every reachable task whatever a late arrival costs. Against the tuned competitors, the cost-aware fallback was on par with the proportional buffer and ahead of the tuned safety factor at half a task reward, and ahead of both at one task reward. At two task rewards, the proportional buffer tuned for this cost was better by 1.02 pp. A fallback task then needs a probability of at least 0.67 of being on time, and the delay that it causes to later tasks on its route, which the rule ignores, weighs more. The competitors, however, have to be tuned again for every cost. Tuned for one task reward,

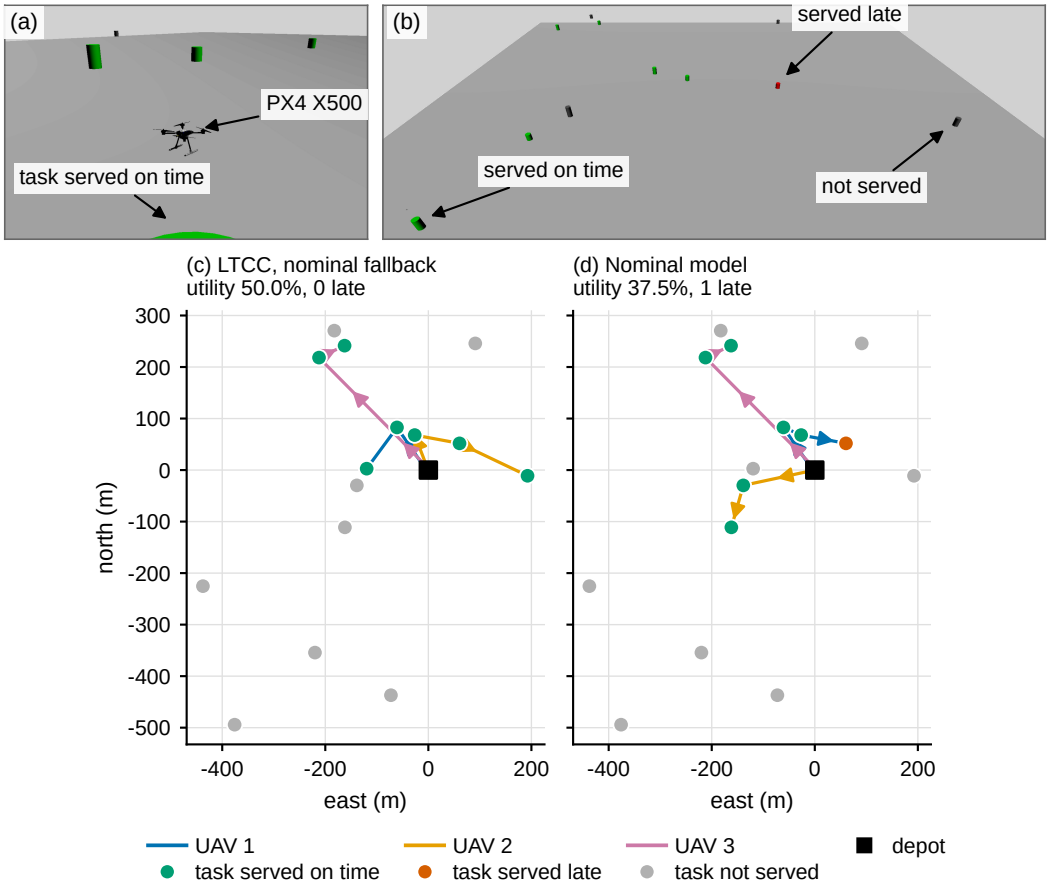



**Figure 4:** Illustrative software-in-the-loop flight on a development mission. (a), (b) Gazebo scene. (c) LTCC with nominal fallback and (d) nominal model: trajectories recorded by PX4.

Table 6
Study 5: score $u_\lambda$ (%) for a late cost of $\lambda$ task rewards.

| Method | $\lambda = 0$ | 0.5 | 1 | 2 | Late |
|---|---|---|---|---|---|
| Tuned proportional buffer | - | 49.28 | 44.91 | 40.73 | 4.36 |
| LTCC, no fallback, tuned safety factor | - | 47.84 | 44.03 | 39.73 | 3.00 |
| Moment propagation, nominal fallback | 54.57 | 49.14 | 43.72 | 32.87 | 7.29 |
| Moment propagation, no fallback | 45.06 | 44.00 | 42.94 | 40.81 | 1.50 |
| Constant buffer (60 s) | 50.33 | 45.66 | 40.98 | 31.64 | 6.49 |
| Nominal model | 49.63 | 41.85 | 34.07 | 18.50 | 11.08 |
| Conventional planner | 47.84 | 40.56 | 33.27 | 18.71 | 10.25 |
| LTCC, nominal fallback | 54.33 | 48.86 | 43.40 | 32.46 | 7.39 |
| LTCC, cost-aware fallback | 54.33 | 49.49 | 45.81 | 39.71 | 4.96 |

$\lambda = 0$ is the utility. The cost-aware fallback and the tuned competitors run with the late cost of each column. Late: tasks served late per mission at $\lambda = 1$. 270 missions.

the proportional buffer stayed 1.05 pp behind the cost-aware fallback at two task rewards.

In practical units, per 100 tasks the cost-aware fallback served 50.2 on time and 7.3 late, the nominal fallback 51.3 and 10.9, and the nominal model 48.3 and 15.6. The fleet flew 92.2 km per mission instead of 101.5 and 104.5 km, consistent with fewer flights to tasks that were likely to be reached too late.

### 6.5. Comparison with CBBA and moment propagation

Table 7 adds CBBA and moment propagation with the cost-aware fallback. All methods with this fallback were run at one task reward and are scored at every late cost from the same runs, and the last two columns give the late tasks per mission and the planning time per call. With the score $u_1$, LTCC exceeded CBBA by 12.77 pp (95% CI 11.91 to 13.63, H20), and on utility alone by 4.86 pp, or 10.0%.

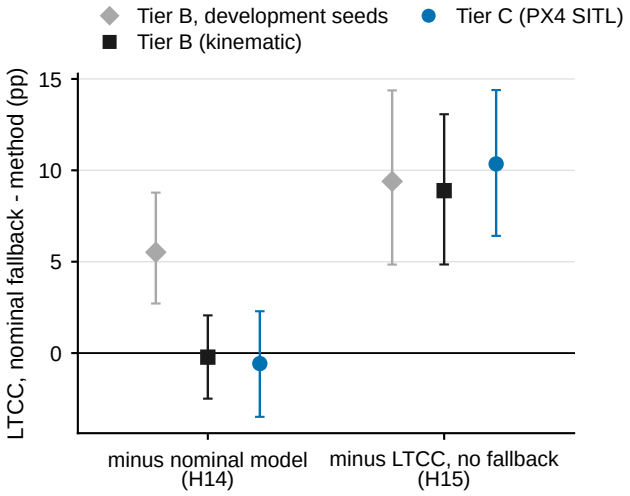



**Figure 5:** Study 4C: utility of LTCC with nominal fallback minus the nominal model (H14) and minus LTCC without fallback (H15), in tier C, in tier B on the same missions and in tier B on development seeds. Error bars are 95% intervals.

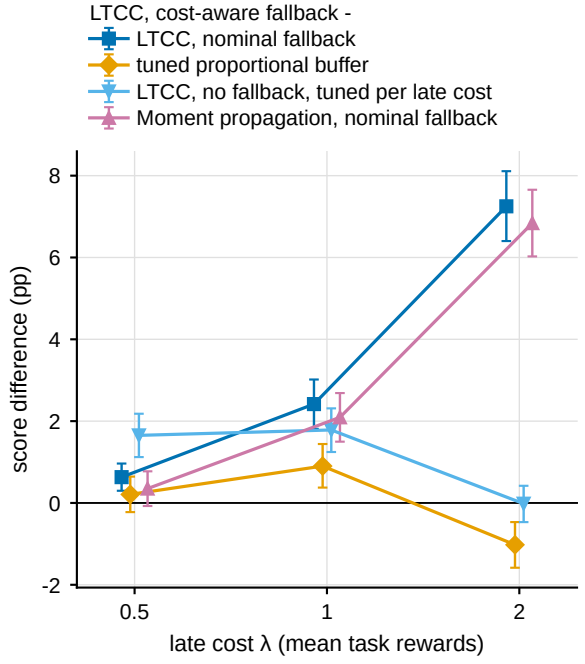



**Figure 6:** Study 5: score of LTCC with the cost-aware fallback minus each method against the late cost. Error bars are 95% intervals over 270 missions.

The difference lies in the planning model rather than in the search. CBBA converged after 2.8 consensus rounds and 5.1 ms per call and reached almost the utility of ALNS on the nominal model. Planned on the LTCC deadlines, it gained 12.12 pp (H21) and came within 0.65 pp of LTCC with 2.1 ms per call. The effective deadlines can thus be handed to an auction-based allocator without changing its bidding rule. The advantage of the cost-aware over the nominal fallback within LTCC, 2.42 pp, replicated H16 on a second set of seeds.

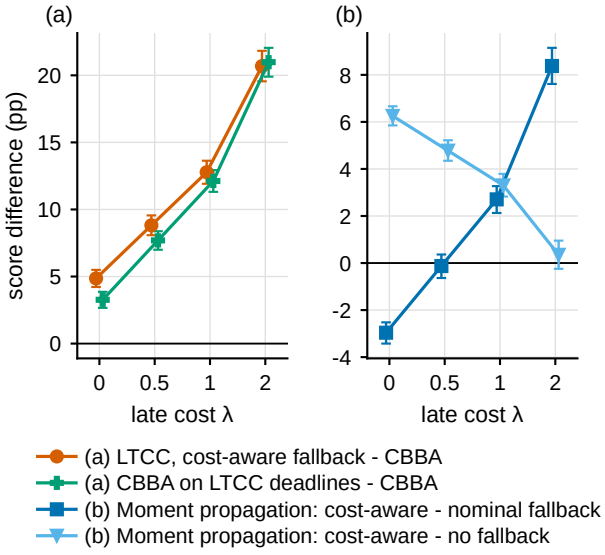



**Figure 7:** Study 6: score differences against the late cost. (a) Comparisons with CBBA. (b) Cost-aware fallback within moment propagation. Error bars are 95% intervals over 270 missions.

Fig. 7 plots the primary differences of study 6 against the late cost, the comparisons with CBBA in panel (a) and the fallbacks within moment propagation in panel (b). Within moment propagation, the cost-aware fallback scored 2.71 pp above the nominal fallback (H22) and 3.30 pp above no fallback (H23) at one task reward. As in the closed form, it gave up some utility when lateness was free, broke even at half a task reward and gained most at two, so the rule is not tied to the closed form. With the cost-aware fallback, moment propagation scored 0.76 pp above the closed form at one task reward and 2.94 pp at two, whereas the closed form was 1.41 pp ahead when lateness was free. Once lateness has a cost, the variance that moment propagation tracks leg by leg pays off.

## 6.6. Late-cost model and errors outside the model

Fig. 8a compares the two cost models of the rule with the proportional buffer tuned for the same score and late cost. Each row is one score and late cost, and a point right of zero favours the cost-aware fallback. At one task reward, matching the rule to the score changed little. With $u_1$, the fixed-cost rule minus the proportional rule was +0.24 [−0.20, +0.67] pp, not a significant difference (H24), and the fixed-cost rule minus the tuned buffer was +1.34 [+0.77, +1.90] pp (H25), which replicates H17 on new seeds. With $u_1^w$, the proportional rule was non-inferior to the buffer tuned for that score (+0.20 [−0.40, +0.79] pp, H26). At twice the cost, each rule did better on its own score. With $u_2$, the fixed-cost rule no longer differed significantly from the tuned buffer, and with $u_2^w$ both rules stayed behind it. The score $u_\lambda$ charges a late arrival the realised mean reward of the mission, whereas the fixed-cost rule assumes the mean reward $\bar{w} = 2.1$ of the generator, because the total reward is not known while tasks are still arriving. Study 7a was therefore

Table 7
Study 6: score $u_\lambda$ (%), late tasks and planning time.

| Method | $\lambda = 0$ | 0.5 | 1 | 2 | Late | ms |
|---|---|---|---|---|---|---|
| CBBA on LTCC deadlines | 51.82 | 48.69 | 45.56 | 39.30 | 4.25 | 2.1 |
| CBBA | 48.56 | 41.00 | 33.44 | 18.32 | 10.57 | 5.1 |
| CBBA, event trigger only | 46.69 | 39.58 | 32.46 | 18.23 | 9.77 | 13.2 |
| Moment propagation, cost-aware fallback | 52.01 | 49.49 | 46.98 | 41.95 | 3.52 | 40.3 |
| Moment propagation, nominal fallback | 54.97 | 49.62 | 44.27 | 33.57 | 7.18 | 40.2 |
| Moment propagation, no fallback | 45.75 | 44.71 | 43.67 | 41.60 | 1.51 | 40.2 |
| Nominal model | 49.55 | 41.76 | 33.98 | 18.42 | 11.02 | 40.1 |
| LTCC, cost-aware fallback | 53.42 | 49.81 | 46.21 | 39.01 | 4.91 | 40.2 |

Methods with the cost-aware fallback run with $\lambda = 1$ and are scored at every $\lambda$ from the same runs. Late: tasks served late per mission; ms: planning time per call. 270 missions.

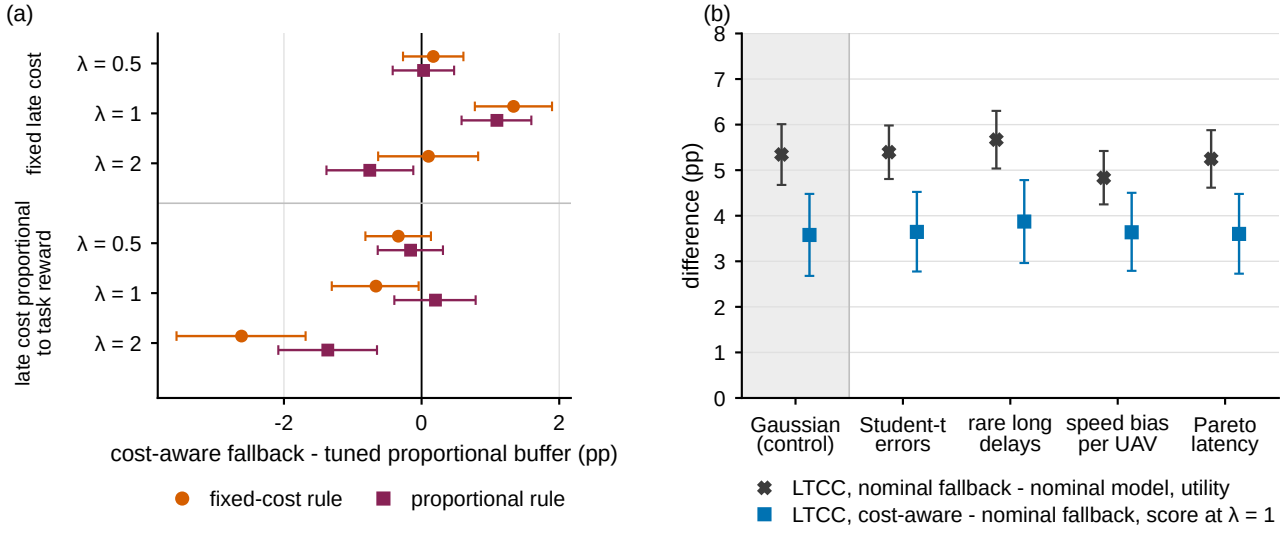



**Figure 8:** Study 7. (a) Cost-aware fallback minus the tuned proportional buffer for each score and late cost. (b) Primary comparisons under the calibrated errors and four departures from the error model. Error bars are 95% intervals over 270 missions.

also scored with $u'_\lambda = U - \lambda \bar{w}\, n_{\text{late}} / R$, which charges exactly the cost that the rule assumes, and the conclusions did not change (Supplementary Table S12).

Fig. 8b shows the two primary comparisons of study 7b under the calibrated errors, shaded as control, and under four departures from the error model. The advantages hardly changed. With the nominal fallback, LTCC served 4.83 to 5.67 pp more than the nominal model (H27), against 5.34 pp under the calibrated errors, and the cost-aware fallback scored 3.60 to 3.87 pp more than the nominal fallback (H28). All eight comparisons were significant. The smallest advantage occurred with a speed bias per UAV, which correlates the errors along a route. With rare long delays, the predicted probability became slightly optimistic in the upper tail, with 92.6% of the tasks served on time against a predicted 93.6% (Supplementary Section S11).

## 7. Discussion

Three findings run through the studies. The first is that the planning model matters more than the control of computation. A margin that grows with the lead time raised the utility by more than any stopping rule or trigger, on three independent sets of seeds, with repair only and with CBBA in place of ALNS. The stopping rule was equivalent to a matched fixed budget, probably because the improvement curve of this ALNS flattens within a few tens of milliseconds, and its main value lies in avoiding the losses of long budgets on slow planners. Periodic replanning helped every method on the nominal model, but only by 0.22 pp with the effective deadlines. Both seem to correct the same accumulated deviations, one before and one after they occur.

The second finding is that the shape of the margin matters more than the way it is computed. The constant buffer was worse than every margin that grows with the lead time, whereas the effective deadline, a tuned proportional buffer and moment propagation reached a similar utility. Because the spread was close to affine in the lead time, a margin proportional to the lead time captures most of it, and under high disturbances the tuned slope nearly equalled the one implied by the calibrated model. The advantages of the closed form are practical. It needs one division per task, no model of the individual legs and no change to the feasibility checks of a heuristic, which is what allowed CBBA to use

it unchanged. Where lateness is costly and the planner can afford to evaluate every leg, moment propagation scored higher.

The third finding is that a probability of meeting the deadline lets the replanner price lateness. A hard chance constraint discards tasks that are still worth attempting, and the nominal fallback recovers them but serves many of them late. Once a late arrival has a cost, the question is no longer only how large the margin should be but which tasks are worth attempting, and this question needs a probability. The break-even rule answers it with another closed-form deadline, without cost-specific tuning, and it behaved in the same way inside moment propagation. A proportional buffer matched it in utility, but once lateness had a cost, it reached a similar score only when it was tuned again for the cost at hand, and it fell behind when it was tuned for another cost.

The two tiers play different roles. The kinematic tier resolves differences of a few percentage points over tens of thousands of missions, whereas a flight with autopilot and vehicle dynamics is costly. Tier B predicted every tier C comparison on the same missions, so the flights support the transfer of the kinematic results. In the software-in-the-loop flights, however, the fallback was confirmed, whereas the advantage of LTCC over the nominal model was not, and this advantage rests on the kinematic tier. Further flight studies can be sized from the kinematic tier with a conservative effect.

These results hold under the conditions studied. The error model is fitted per disturbance level, or online from the tasks of the mission, and the guarantees of Propositions 1 and 2 rest on a Gaussian error model, although the advantages persisted under the four tested departures from it. The cost-aware fallback prices each attempt on its own. A route-level version that also charges the delay a fallback task causes to the tasks after it, for example through a late cost that grows with their number, is a natural next step, in particular for high late costs. The flights used three simulated UAVs, so tests with the cost-aware fallback in the autopilot loop, on embedded hardware and in the field remain to be done.

## 8. Conclusion

This paper proposed LTCC, a multi-UAV replanner that plans against a chance constraint on the finish time of every task. In the tested missions, the spread of the finish-time error was well approximated by an affine function of the lead time, and with this error model the chance constraint reduces exactly to an effective deadline per task, so fast heuristics are reused unchanged. A cost-aware fallback uses the probability of meeting the deadline to decide which given-up tasks to attempt, again in closed form and without cost-specific tuning. In seven studies on held-out missions, LTCC served more reward than the nominal model, a constant buffer, a conventional planner and CBBA, and as much as a tuned proportional buffer and moment propagation. With a late cost of one task reward, the cost-aware fallback scored above the nominal fallback and above a buffer tuned for that cost, on two sets of seeds. In PX4 software-in-the-loop flights, the fallback raised the utility by 56%, as the kinematic tier had predicted. Future work will charge the delay that a fallback task causes on its route, fly the cost-aware fallback with PX4, run it inside a distributed allocator and test the replanner on embedded hardware.

## CRediT authorship contribution statement

**Cao-Phuc Ha:** Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Visualization, Writing - original draft. **Vu Minh Anh Nguyen:** Validation, Investigation, Visualization, Writing - review & editing. **Duong Thanh Tran:** Methodology, Formal analysis, Validation, Writing - review & editing. **Phu-Nguyen Le:** Conceptualization, Methodology, Supervision, Resources, Project administration, Writing - review & editing.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. No author received financial support for the research, authorship or publication of this article.

## Data availability

The planner, both simulation tiers, the experiment configurations with their SHA-256 hashes, the written protocols with every deviation, the raw results of all held-out runs and the scripts that produce every number, table and figure of this paper are available at https://github.com/highptest1-36/AnytimeSwarm.

## Acknowledgements

The authors would like to express their sincere appreciation to the Vietnam Aerospace University for supporting this research during the 2026-2027 academic year.

## References

[1] B. P. Gerkey, M. J. Matarić, A formal analysis and taxonomy of task allocation in multi-robot systems, Int. J. Robot. Res. 23 (9) (2004) 939-954. doi:10.1177/0278364904045564.

[2] H. N. Psaraftis, M. Wen, C. A. Kontovas, Dynamic vehicle routing problems: Three decades and counting, Networks 67 (1) (2016) 3-31. doi:10.1002/net.21628.

[3] N. Soeffker, M. W. Ulmer, D. C. Mattfeld, Stochastic dynamic vehicle routing in the light of prescriptive analytics: A review, Eur. J. Oper. Res. 298 (3) (2022) 801-820. doi:10.1016/j.ejor.2021.07.014.

[4] G. M. Skaltsis, H.-S. Shin, A. Tsourdos, A review of task allocation methods for UAVs, J. Intell. Robot. Syst. 109 (4) (2023) 76. doi:10.1007/s10846-023-02011-0.

[5] H.-L. Choi, L. Brunet, J. P. How, Consensus-based decentralized auctions for robust task allocation, IEEE Trans. Robot. 25 (4) (2009) 912-926. doi:10.1109/TRO.2009.2022423.

[6] S. Ropke, D. Pisinger, An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows, Transp. Sci. 40 (4) (2006) 455-472. doi:10.1287/trsc.1050.0135.

[7] A. Agra, M. Christiansen, R. Figueiredo, L. M. Hvattum, M. Poss, C. Requejo, The robust vehicle routing problem with time windows, Comput. Oper. Res. 40 (3) (2013) 856-866. doi:10.1016/j.cor.2012.10.002.

[8] J. F. Ehmke, A. M. Campbell, T. L. Urban, Ensuring service levels in routing problems with time windows and stochastic travel times, Eur. J. Oper. Res. 240 (2) (2015) 539-550. doi:10.1016/j.ejor.2014.06.045.

[9] B. Rostami, G. Desaulniers, F. Errico, A. Lodi, Branch-price-and-cut algorithms for the vehicle routing problem with stochastic and correlated travel times, Oper. Res. 69 (2) (2021) 436-455. doi:10.1287/opre.2020.2037.

[10] C. Cheng, Y. Adulyasak, L.-M. Rousseau, Robust drone delivery with weather information, Manuf. Serv. Oper. Manag. 26 (4) (2024) 1402-1421. doi:10.1287/msom.2022.0339.

[11] S. S. Ponda, L. B. Johnson, J. P. How, Distributed chance-constrained task allocation for autonomous multi-agent teams, in: Proceedings of the American Control Conference (ACC), 2012, pp. 4528-4533. doi:10.1109/ACC.2012.6315626.

[12] Z. Gao, M. Zheng, Y. Mei, A. Zheng, H. Zhong, Distributionally robust chance-constrained task assignment for heterogeneous UAVs with time windows under uncertain fuel consumption, Drones 9 (9) (2025) 633. doi:10.3390/drones9090633.

[13] O. Jabali, R. Leus, T. Van Woensel, T. de Kok, Self-imposed time windows in vehicle routing problems, OR Spectrum 37 (2) (2015) 331-352. doi:10.1007/s00291-013-0348-1.

[14] S. Zilberstein, Using anytime algorithms in intelligent systems, AI Mag. 17 (3) (1996) 73-83. doi:10.1609/aimag.v17i3.1232.

[15] J. Svegliato, K. H. Wray, S. Zilberstein, Meta-level control of anytime algorithms with online performance prediction, in: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI-18), IJCAI Organization, 2018, pp. 1499-1505. doi:10.24963/ijcai.2018/208.

[16] J. W. Herrmann, Metareasoning for Robots: Adapting in Dynamic and Uncertain Environments, Synthesis Lectures on Computer Science, Springer, Cham, 2023. doi:10.1007/978-3-031-32237-2.

[17] D. Taş, N. Dellaert, T. van Woensel, T. de Kok, Vehicle routing problem with stochastic travel times including soft time windows and service costs, Comput. Oper. Res. 40 (1) (2013) 214-224. doi:10.1016/j.cor.2012.06.008.

[18] F. Bomboi, C. Buchheim, J. Pruente, On the stochastic vehicle routing problem with time windows, correlated travel times, and time dependency, 4OR 20 (2) (2022) 217-239. doi:10.1007/s10288-021-00476-z.

[19] M. Boddy, T. L. Dean, Deliberation scheduling for problem solving in time-constrained environments, Artif. Intell. 67 (2) (1994) 245-285. doi:10.1016/0004-3702(94)90054-X.

[20] L. Meier, D. Honegger, M. Pollefeys, PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms, in: Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2015, pp. 6235-6240. doi:10.1109/ICRA.2015.7140074.

[21] N. Koenig, A. Howard, Design and use paradigms for Gazebo, an open-source multi-robot simulator, in: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Vol. 3, 2004, pp. 2149-2154. doi:10.1109/IROS.2004.1389727.

[22] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot Operating System 2: Design, architecture, and uses in the wild, Sci. Robot. 7 (66) (2022) eabm6074. doi:10.1126/scirobotics.abm6074.

[23] H. Chakraa, F. Guérin, E. Leclercq, D. Lefebvre, Optimization techniques for multi-robot task allocation problems: Review on the state-of-the-art, Robot. Auton. Syst. 168 (2023) 104492. doi:10.1016/j.robot.2023.104492.

[24] D. Rojas Viloria, E. L. Solano-Charris, A. Muñoz-Villamizar, J. R. Montoya-Torres, Unmanned aerial vehicles/drones in vehicle routing problems: A literature review, Int. Trans. Oper. Res. 28 (4) (2021) 1626-1657. doi:10.1111/itor.12783.

[25] M. M. Solomon, Algorithms for the vehicle routing and scheduling problems with time window constraints, Oper. Res. 35 (2) (1987) 254-265. doi:10.1287/opre.35.2.254.

[26] P. Vansteenwegen, W. Souffriau, D. Van Oudheusden, The orienteering problem: A survey, Eur. J. Oper. Res. 209 (1) (2011) 1-10. doi:10.1016/j.ejor.2010.03.045.

[27] S. Wang, W. Sun, M. Huang, An adaptive large neighborhood search for the multi-depot dynamic vehicle routing problem with time windows, Comput. Ind. Eng. 191 (2024) 110122. doi:10.1016/j.cie.2024.110122.

[28] W. Wang, M. Lv, L. Ru, B. Lu, S. Hu, X. Chang, Multi-UAV unbalanced targets coordinated dynamic task allocation in phases, Aerospace 9 (9) (2022) 491. doi:10.3390/aerospace9090491.

[29] L. Zhao, S. Li, Z. Tan, A. Hawbani, S. Timotheou, K. Yu, A multi-UAV cooperative task scheduling in dynamic environments: Throughput maximization, IEEE Trans. Comput. 74 (2) (2025) 442-454. doi:10.1109/TC.2024.3483636.

[30] S. Choudhury, J. K. Gupta, M. J. Kochenderfer, D. Sadigh, J. Bohg, Dynamic multi-robot task allocation under uncertainty and temporal constraints, Auton. Robots 46 (1) (2022) 231-247. doi:10.1007/s10514-021-10022-9.

[31] H. Huang, Z. Jiang, T. Yan, Y. Bai, Dynamic task allocation for heterogeneous multi-UAVs in uncertain environments based on 4DI-GWO algorithm, Drones 8 (6) (2024) 236. doi:10.3390/drones8060236.

[32] C. Qian, B. Fu, Uncertainty-gated distributed dynamic task allocation for multi-UAV systems under intermittent communication, Drones 10 (10) (2026) 782. doi:10.3390/drones10100782.

[33] S.-Y. Hwang, S. Oh, J.-C. Lee, S. Lee, J.-H. Lee, H.-J. Hwang, Assignment-preserving replanning for dynamic task allocation in multi-UAV systems, Appl. Sci. 16 (19) (2026) 9414. doi:10.3390/app16199414.

[34] Y. Shen, T. Hu, Y. Wan, H. Sun, M. Tan, Edge-responsive local-repair particle swarm optimization for dynamic spatiotemporal task allocation of IoT-connected UAV swarms, IEEE Access 14 (2026) 115169-115190. doi:10.1109/ACCESS.2026.3717031.

[35] Y. Cong, Y. Li, X. Zhu, Z. Wang, An event-triggered path-based bid assignment method for energy-aware dynamic multi-UAV task allocation, Drones 10 (9) (2026) 684. doi:10.3390/drones10090684.

[36] Á. Calvo, J. Capitán, Heterogeneous multirobot task allocation for long-endurance missions in dynamic scenarios, IEEE Trans. Robot. 41 (2025) 6494-6513. doi:10.1109/TRO.2025.3626651.

[37] M. A. Ali, U. Athar, A. Maqsood, SITL validation of federated reinforcement learning for priority-based dynamic task allocation in UAV swarms, Expert Syst. 43 (12) (2026) e70459. doi:10.1111/exsy.70459.

[38] L. Zhou, P. Tokekar, Risk-aware submodular optimization for multirobot coordination, IEEE Trans. Robot. 38 (5) (2022) 3064-3084. doi:10.1109/TRO.2022.3158227.

[39] M. E. Bruni, S. Khodaparasti, G. Perboli, The drone latency location routing problem under uncertainty, Transp. Res. Part C Emerg. Technol. 156 (2023) 104322. doi:10.1016/j.trc.2023.104322.

[40] J. Wang, G. Jia, Z. Guo, Z. Hou, Impact of uncertain flight time on heterogeneous UAVs' task planning with temporal constraints, Aerospace 11 (3) (2024) 214. doi:10.3390/aerospace11030214.

[41] B. Lacerda, A. Gautier, A. Rutherford, A. Stephens, C. Street, N. Hawes, Decision-making under uncertainty for multi-robot systems, AI Commun. 35 (4) (2022) 433-441. doi:10.3233/AIC-220118.

[42] F. Errico, G. Desaulniers, M. Gendreau, W. Rei, L.-M. Rousseau, A priori optimization with recourse for the vehicle routing problem with hard time windows and stochastic service times, Eur. J. Oper. Res. 249 (1) (2016) 55-66. doi:10.1016/j.ejor.2015.07.027.

[43] X. Li, P. Tian, S. C. H. Leung, Vehicle routing problems with time windows and stochastic travel and service times: Models and algorithm, Int. J. Prod. Econ. 125 (1) (2010) 137-145. doi:10.1016/

j.ijpe.2010.01.013.

[44] C. E. Clark, The greatest of a finite set of random variables, Oper. Res. 9 (2) (1961) 145-162. doi:10.1287/opre.9.2.145.

[45] O. Dukkanci, B. Y. Kara, T. Bektaş, Minimizing energy and cost in range-limited drone deliveries with speed optimization, Transp. Res. Part C Emerg. Technol. 125 (2021) 102985. doi:10.1016/j.trc.2021.102985.

[46] Y. Sung, L. P. Kaelbling, T. Lozano-Pérez, Learning when to quit: Meta-reasoning for motion planning, in: Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2021, pp. 4692-4699. doi:10.1109/IROS51168.2021.9636864.

[47] M. Cashmore, A. Coles, B. Cserna, E. Karpas, D. Magazzeni, W. Ruml, Temporal planning while the clock ticks, in: Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS), Vol. 28, AAAI Press, 2018, pp. 39-46. doi:10.1609/icaps.v28i1.13878.

[48] S. S. Shperberg, A. Coles, E. Karpas, W. Ruml, S. E. Shimony, Situated temporal planning using deadline-aware metareasoning, in: Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS), Vol. 31, AAAI Press, 2021, pp. 340-348. doi:10.1609/icaps.v31i1.15979.

[49] M. W. P. Savelsbergh, The vehicle routing problem with time windows: Minimizing route duration, ORSA J. Comput. 4 (2) (1992) 146-154. doi:10.1287/ijoc.4.2.146.

[50] S. T. W. Mara, R. Norcahyo, P. Jodiawan, L. Lusiantoro, A. P. Rifai, A survey of adaptive large neighborhood search algorithms and applications, Comput. Oper. Res. 146 (2022) 105903. doi:10.1016/j.cor.2022.105903.

[51] P. McCullagh, J. A. Nelder, Generalized Linear Models, 2nd Edition, Chapman and Hall, London, 1989. doi:10.1007/978-1-4899-3242-6.

[52] A. M. Law, Simulation Modeling and Analysis, 5th Edition, McGraw-Hill Education, New York, 2015.

[53] L. B. Johnson, H.-L. Choi, S. S. Ponda, J. P. How, Allowing non-submodular score functions in distributed task allocation, in: Proceedings of the 51st IEEE Conference on Decision and Control (CDC), 2012, pp. 4702-4708. doi:10.1109/CDC.2012.6425867.

[54] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman and Hall, New York, 1993. doi:10.1007/978-1-4899-4541-9.

[55] S. Holm, A simple sequentially rejective multiple test procedure, Scand. J. Stat. 6 (2) (1979) 65-70.